Activation Steering for Small Language Models: From Mid-Layer Sweet Spots to Dynamic Multi-Agent Orchestration

Arthur Edmond[1]

[1]LLM Engineer, Swapn

March 14, 2026

Abstract

We present a comprehensive study of **activation steering** for improving task-specific behavior in small language models (SLMs), from mechanistic analysis to practical multi-agent orchestration. Using Qwen3-4B and Qwen3-0.6B as primary models, we (1) analyze tokenizer fragmentation across four models, (2) extract contrastive steering vectors via SAE-inspired methodology, (3) perform a layer×coefficient sweep identifying mid-layer sweet spots, (4) validate on base (non-instruct) models, (5) evaluate on GSM8K math reasoning, (6) quantify effects via KL divergence and sampling diversity, (7) build a dynamic multi-agent steering orchestrator, and (8) test on SWE-bench Verified with RAG-augmented patch generation.

Our key findings are: (a) **mid-layer steering (layers 15–18) achieves 100% behavioral change** at moderate coefficients ($\alpha = 30$), while late layers (33–35) are rigid—an *architectural* property independent of instruction tuning; (b) **base models are more steerable but fragile**—effective at $\alpha = 30$ but degenerate at $\alpha \geq 60$; (c) **steering on a 0.6B SLM boosts GSM8K accuracy by +16pp** under zero-shot CoT (46%→62%), but **degrades under 5-shot** (−8pp), revealing destructive interference between steering and few-shot prompting; (d) **KL divergence confirms a 3-order-of-magnitude gap** between mid-layer (1–48 bits) and late-layer (<0.01 bits) steering; (e) a **dynamic steering orchestrator** that switches domain-specific vectors per plan step achieves 4–7× more domain-relevant outputs than baseline; (f) on SWE-bench Verified, RAG-augmented steering produces patches with **68% valid file paths** (vs. 0% without RAG), with 95% of instances generating syntactically valid diffs. We present a validated architecture for training-free enhancement of SLM agents via sequential steering with adaptive coefficients.

Keywords: activation steering, small language models, mechanistic interpretability, multi-agent orchestration, SWE-bench, dynamic steering

1 Introduction

Small Language Models (SLMs) with 0.6–3B parameters are increasingly deployed as agents that must extract structured information and execute tool calls. However, their limited capacity makes them sensitive to how tasks are framed at inference time. Two families of techniques compete for improving their task-specific behavior:

1. **Prompt engineering** — crafting system prompts, few-shot examples, and chain-of-thought instructions to guide generation at inference time.

2. **Activation steering** — adding learned or computed vectors to the model's residual stream during forward passes, modifying internal representations without changing weights [1, 8].

Recent work has shown that per-layer steering vectors trained via reinforcement learning can match fully fine-tuned models on mathematical reasoning tasks, adding only 0.0016% parameters [2]. Separately, budget guidance techniques demonstrate training-free control of reasoning length via token-level distribution modeling [3]. However, these approaches have primarily been studied on reasoning benchmarks — their effectiveness in agentic tool-use contexts remains unexplored.

Contributions. This paper makes seven contributions:

- A comparative tokenizer analysis across four models revealing universal fragmentation of structured expressions (section 3).
- SAE-inspired contrastive feature extraction with logit lens validation (section 4).
- A comprehensive steering sweep across 11 layers and 6 coefficients, identifying mid-layer steering (layers 15–18) as the effective intervention point (section 5).
- Comparative steering on the base (non-instruct) model, revealing higher malleability but greater fragility (section 6).
- Zero-shot steering synergy on GSM8K (+16pp) contrasted with destructive 5-shot interference (section 9).
- A dynamic multi-agent steering orchestrator with domain-specific vector switching (section 12.3).
- RAG-augmented SWE-bench evaluation showing 0%→68% path validity via steering + retrieval (**??**).

2 Related Work

2.1 Activation Steering

Turner et al. [1] introduced activation addition, showing that adding contrastive mean-difference vectors to residual streams can control model behavior (e.g., shifting sentiment or honesty). Gao et al. [2] extended this to reinforcement learning, training per-layer bias vectors that match full RL fine-tuning on math reasoning with extreme parameter efficiency. Li et al. [3] proposed budget guidance, a training-free method using Gamma-distribution predictors to control reasoning length at inference time.

2.2 Mechanistic Interpretability

Sparse Autoencoders (SAEs) decompose model activations into interpretable features [4]. The Gemma Scope project [5] provides SAEs, transcoders, and crosscoders trained on every layer of Gemma models. Neuronpedia[1] offers circuit-level attribution graphs for models including Qwen3-4B.

2.3 Prompt Engineering for Tool Use

Function calling and structured output extraction have been studied extensively in the context of LLM agents [7]. Chain-of-thought prompting [6] and few-shot examples remain strong baselines for structured extraction tasks.

[1] https://www.neuronpedia.org

3 Tokenizer Analysis

Tokenization quality directly impacts downstream model behavior: a tokenizer that fragments semantic units distributes information across multiple positions in the residual stream, potentially diluting features that steering vectors target.

3.1 Experimental Setup

We compare four tokenizers on nine prompt categories designed for agentic calendar tasks:

Table 1: Tokenizer properties.

Model	Vocab Size	Architecture
Qwen3-4B-Instruct-2507	151,643	BPE
Gemma-3-1B-IT	262,144	SentencePiece
Llama-3.2-3B-Instruct	128,000	BPE
Phi-3-mini-4k-instruct	32,000	SentencePiece

Our test suite includes simple and complex calendar requests in French and English, relative and absolute temporal expressions, JSON tool calls, and system prompts.

3.2 Results

3.2.1 Token Counts and Compression

Table 2: Token counts across prompt categories. Lower is better.

Prompt	Qwen3	Gemma-3	Llama-3.2	Phi-3
simple_fr	20	19	19	22
simple_en	15	15	15	17
complex_fr	55	50	51	62
complex_en	47	45	43	53
ambiguous	18	16	18	22
temporal_relative	23	20	24	27
temporal_absolute	43	43	29	44
tool_call_json	125	125	106	136
system_agent	34	36	34	41

Llama-3.2-3B achieves the best compression on both temporal expressions (29 tokens vs. 43–44 for ISO 8601 dates) and JSON tool calls (106 vs. 125–136). This is attributable to its BPE vocabulary including multi-digit numbers (‘‘24’’, ‘‘202’’) and code-oriented subwords (‘‘`_calendar_event`’’ as a single token).

3.2.2 Quality Scoring

We designed a composite scoring function evaluating four dimensions relevant to agentic calendar use:

Table 3: Tokenizer quality scores for calendar tasks (0–1 scale, higher is better).

Model	Temporal	Semantic	JSON	Multilingual	Overall
Qwen3-4B	0.205	0.680	0.288	0.802	0.483
Gemma-3-1B	0.226	0.722	0.288	0.845	0.511
Llama-3.2-3B	0.316	0.726	0.375	0.816	**0.551**
Phi-3-mini	0.173	0.580	0.249	0.814	0.438

Key finding: All tokenizers score poorly on temporal integrity (< 0.32). ISO 8601 timestamps are universally fragmented into individual characters, creating a distributed representation that complicates feature extraction for steering.

4 Feature Extraction via Contrastive Activations

4.1 Methodology

Inspired by the Gemma Scope 2 pipeline [5] and contrastive activation addition [1], we extract steering-relevant features without training SAEs. Our approach:

1. Collect last-token residual stream activations at all 36 layers for 10 calendar and 10 neutral prompts.
2. Compute per-layer mean activations for each group.
3. Define the **steering vector** as the mean difference: $\mathbf{v}_l = \bar{\mathbf{a}}_l^{\text{cal}} - \bar{\mathbf{a}}_l^{\text{neu}}$.
4. Rank layers by $\|\mathbf{v}_l\|_2$ (separation magnitude) and cosine distance.

4.2 Layer Importance

Table 4: Top-10 layers by steering vector L2 norm.

Rank	Layer	L2 Norm	Cosine Distance
1	35	361.00	0.151
2	34	218.63	0.075
3	33	179.38	0.094
4	32	162.50	0.104
5	31	143.63	0.113
6	30	126.19	0.120
7	29	110.31	0.130
8	28	100.44	0.151
9	27	89.50	0.164
10	26	79.50	0.168

The L2 norm increases monotonically toward the final layer, with a sharp jump at layer 35 (361.0 vs. 218.6 at layer 34). This is consistent with the "representation engineering" hypothesis: later layers encode increasingly task-specific, high-level semantic features.

4.3 Logit Lens Analysis

We project the top-3 steering vectors through the unembedding matrix ($\mathbf{W}_U$) after applying RMSNorm weights:

$$\text{logits}_l = \mathbf{W}_U \cdot (\mathbf{w}_{\text{norm}} \odot \mathbf{v}_l)$$

Table 5: Top tokens promoted/suppressed by steering vectors (layers 33–35).

Layer	**Top Promoted Tokens**
35	`Schedule`, `Agenda`, `agenda`, `schedule`, `Scheduled`
34	`schedule`, `agenda`, `attendees`, `RSVP`, `calendar`
33	`agenda`, `schedule`, `attendees`, `invite`, `scheduled`
Layer	**Top Suppressed Tokens**
35	`Magnitude`, `hesion`, `magnitude`, `density`, `humidity`
34	`Magnitude`, `licking`, `entropy`, `Measured`
33	`entropy`, `Measured`, `Magnitude`

Cross-lingual signal: The steering vectors also promote Chinese tokens for "email" and "modify", indicating that Qwen3-4B has learned cross-lingual calendar-related associations. This is notable given the model's multilingual training on 119 languages.

5 Steering Vector Application

5.1 Methodology

We apply steering vectors using activation addition [1]:

$$\mathbf{h}'_l = \mathbf{h}_l + \alpha \cdot \frac{\mathbf{v}_l}{\|\mathbf{v}_l\|}$$

where $\alpha \in \{0, 10, 30, 60, 100, 200\}$ and $\mathbf{h}_l$ is the residual stream at layer l. We sweep across 11 layers ($l \in \{5, 10, 15, 18, 20, 22, 25, 28, 30, 33, 35\}$) on six prompts: 2 calendar (FR/EN), 2 ambiguous (FR/EN), and 2 non-calendar (FR/EN).

5.2 Initial Finding: Late-Layer Rigidity

Applying the layer-35 vector (highest L2 norm, 361.0) produces **zero behavioral change** across all coefficients up to $\alpha = 50$. All responses are character-for-character identical to baseline. This motivates the broader layer sweep.

5.3 Mid-Layer Steering Sweet Spot

Table 6: Change rate (%) vs. baseline across layers and coefficients on Qwen3-4B-Instruct. Bold indicates 100% change rate with coherent outputs.

	Steering Coefficient (α)					
Layer	0	10	30	60	100	200
5	0	33	33	100†	67†	67†
10	0	17	67	83	67	100
15	**0**	**17**	**100**	**100**	**83**	**100**
18	**0**	**33**	**100**	**100**	**100**	**67**
20	0	17	33	100	100	67
22	0	17	17	50	100	67
25	0	0	17	33	50	83
30	0	0	17	17	17	50
33	0	0	17	17	17	17
35	0	0	0	0	0	17

† Layer 5 at high α produces degenerate (empty) outputs.

Key result: Layers 15–18 at $\alpha = 30$ achieve 100% change rate with coherent, well-formed responses. This is the *steering sweet spot*—representations at these layers are transitioning from syntactic to semantic and remain malleable enough for intervention.

Qualitative effect: Non-calendar prompts get reinterpreted as calendar tasks. For instance, *"Explain how transformers work"* becomes *"Let's schedule a meeting to discuss transformers"*—the steering vector genuinely redirects the model's intent, not just its surface tokens.

A clear **gradient of rigidity** emerges: late layers (30–35) require 3–10× higher coefficients for the same effect, and even then produce only partial changes. This confirms the hypothesis that instruction tuning "freezes" late-layer representations.

6 Base Model Steering

6.1 Setup

To disentangle the effect of instruction tuning, we repeat the steering experiment on **Qwen3-4B** (base, non-instruct). We extract fresh contrastive vectors using completion-style prompts (no chat template) with 10 calendar and 10 neutral examples.

6.2 Results

Table 7: Calendar score (0–1) for base model steering. Higher = stronger calendar signal. Baseline avg = 0.70.

	Steering Coefficient (α)					
Layer	0	10	30	60	100	200
10	0.70	0.73	0.70	0.73	0.23	0.00
15	**0.70**	**0.70**	**0.97**	**0.33**	**0.03**	**0.00**
18	0.70	0.73	0.73	0.13	0.00	0.00
20	0.70	0.73	0.73	0.03	0.00	0.00
25	0.70	0.73	0.73	0.73	0.77	0.00
30	0.70	0.70	0.70	0.73	0.73	0.70
35	0.70	0.70	0.70	0.70	0.73	0.77

Three findings emerge:

1. **Layer 15 at $\alpha = 30$ is the optimal point**: calendar score jumps from 0.70 to 0.97 (+0.27), matching the instruct model's sweet spot layer.

2. **Base models are much more fragile**: at $\alpha \geq 60$, scores collapse to near-zero (degenerate output). The instruct model tolerates $\alpha = 200$ at mid-layers without complete degeneration.

3. **Late-layer rigidity is architectural**: layers 30–35 resist steering on both instruct and base models, ruling out instruction tuning as the sole cause. This is a property of the transformer architecture itself—late residual streams are too committed to the output distribution.

7 Budget Guidance

Inspired by Li et al. [3], we implement a simplified budget guidance mechanism: a Gamma-distribution predictor that biases the EOS token logit as generation approaches a token budget, using a sigmoid urgency ramp.

7.1 Results

We test budgets of 32, 64, 128, 256, and 512 tokens on 5 calendar prompts:

Table 8: Budget guidance results on Qwen3-4B-Instruct. Baseline avg: 75 tokens.

Budget	Avg Tokens	Valid	Within Budget	Savings
∞ (baseline)	74.6	100%	100%	0%
32	74.6	100%	0%	0%
64	74.6	100%	0%	0%
128	74.6	100%	100%	0%
256	74.6	100%	100%	0%
512	74.6	100%	100%	0%

Budget guidance has zero effect. The instruct model already generates near-minimal outputs ($\sim$75 tokens of compact JSON). Even the most aggressive budget (32 tokens) fails to further compress the output—the model's greedy decoding path is already optimal for structured extraction. This contrasts with reasoning-heavy tasks (e.g., GSM8K) where models produce verbose chain-of-thought before answering, and where budget guidance can yield significant savings [3].

Implication: Budget guidance targets *thinking overhead*, not output verbosity. For tasks where instruction tuning has already eliminated unnecessary reasoning tokens, it adds no value.

8 Neuronpedia Feature Mapping

We explore Qwen3-4B's SAE features via Neuronpedia's transcoder dictionary (163,840 features per layer) using three strategies: explanation keyword search, random sampling, and targeted key-layer scans.

8.1 Results

- **Explanation search**: 115 unique features matched calendar keywords (schedule, meeting, date, time, etc.) across layers 20–35.

- **Random sampling**: only 9 out of 450 sampled features (2%) were calendar-related, confirming extreme sparsity.
- **Feature distribution**: calendar features concentrate in later layers (20–35), consistent with our contrastive analysis. The highest-layer features show the most specific explanations (e.g., "scheduling appointments" vs. generic "time references").

The 0.07% density of calendar features (9/450 random samples × 163,840 features/layer) implies approximately 115 calendar-relevant features per layer—a tiny fraction of the total, but sufficient to encode the task-specific behavior observed in our steering experiments.

9 SLM Steering on GSM8K

To test whether steering generalizes beyond calendar tasks and instruction-tuned models, we run a parallel experiment on **Qwen3-0.6B** (28 layers, 1024 hidden dim) using math reasoning (GSM8K) as the target task.

9.1 Setup

We extract contrastive vectors from 10 "math reasoning" prompts vs. 10 neutral prompts. We evaluate on 10 GSM8K problems using three prompt strategies (zero-shot, CoT, few-shot) and steering across 6 layers × 5 coefficients, on both the instruct and base variants.

9.2 Results

Table 9: GSM8K accuracy (%) on Qwen3-0.6B. Best steering result per layer in bold.

	Steering Coefficient (α)				
Layer	0	10	30	60	100
Instruct model (baseline: 20% zero-shot)					
5	20	0	0	0	0
15	20	0	0	0	0
18	20	10	20	10	0
25	20	20	10	**30**	10
Base model (baseline: 20% few-shot)					
5	20	0	0	0	0
15	20	20	0	10	0
18	20	20	**30**	20	0
20	20	20	20	30	**40**
25	20	20	30	30	30

Three findings emerge:

1. **Steering is more effective on the base model**: Layer 20 at $\alpha = 100$ **doubles** GSM8K accuracy (20%→40%). The instruct model only achieves +10% at one configuration (Layer 25, $\alpha = 60$).
2. **The sweet spot shifts with model depth**: On the 0.6B (28 layers), optimal steering occurs at layers 18–25 (64–89% depth), compared to layers 15–18 (42–50%) on the 4B (36 layers). This suggests a **relative depth hypothesis**: the steering sweet spot follows the syntactic→semantic transition, which scales with total layer count.

3. **SLMs are extremely fragile**: Early layers (5–10) at any non-zero coefficient universally destroy accuracy, and CoT prompting actually *hurts* the 0.6B model (10% vs. 20% zero-shot).

9.3 Validation via lm-eval (n=50)

To validate our pilot results with a standardized evaluation, we run the EleutherAI `lm-eval` harness [12] on two GSM8K task variants: the standard **5-shot** task and the **zero-shot CoT** task (`gsm8k_cot_zeroshot`, prompt: "Let's think step by step"). We subclass `HFLM` to inject steering hooks during both `_model_call` and `_model_generate`.

Table 10: GSM8K lm-eval results (n=50). 5-shot vs. zero-shot CoT reveal opposite steering effects.

		5-shot		**Zero-shot CoT**	
Model	**Condition**	**Strict**	**Flexible**	**Strict**	**Flexible**
Instruct	Baseline	48%	48%	38%	46%
Instruct	Steered	44% (−4)	40% (−8)	40% (+2)	**62% (+16)**
Base	Baseline	48%	48%	36%	28%
Base	Steered	26% (−22)	34% (−14)	8% (−28)	22% (−6)

Instruct steered: L25@α=60. Base steered: L20@α=100.

The 5-shot and zero-shot results tell **opposite stories**, revealing a critical interaction between steering and prompting strategy:

1. **Zero-shot CoT + steering = synergy on instruct.** The steered instruct model achieves **62% flexible-extract** (+16pp over baseline), the best result across all conditions. The strict match remains stable (38%→40%), indicating that steering improves *reasoning* without improving *format compliance*—the model finds correct answers more often but doesn't always use the "The answer is X" template.

2. **5-shot + steering = destructive interference.** Under 5-shot evaluation, steering *degrades* both models. The few-shot exemplars already provide strong reasoning scaffolding (∼350 context tokens); the steering vector is redundant with, and interferes with, this guidance. The steered base model drops from 48%→26% strict match.

3. **Base model at α=100 degenerates regardless.** The base model shows degradation in both evaluation protocols. This confirms α=100 is too aggressive for the 0.6B base model (consistent with the narrow stability window observed in section 6). Lower coefficients (α=30–60) would likely yield improvements.

4. **Generation pathology at high α.** Steered base model generation took 3.3× longer under 5-shot (66 vs. 20 minutes), suggesting the steering vector pushes the model into longer, erratic generation paths before hitting a stop token.

Implication for dynamic steering: Steering coefficients must be *adaptive* to the prompting context. A coefficient that helps under zero-shot conditions can hurt when few-shot exemplars provide directional guidance. This motivates context-aware selection: $\alpha = f(n_{\text{few-shot}}, \text{model_type}, \text{task_domain}$ a key design requirement for the dynamic multi-agent steering architecture discussed in section 12.

10 Sampling-Based Steering Analysis

Previous experiments used greedy decoding (T=0), which collapses all probability mass onto the argmax token. To detect subtler steering effects, we analyze the full logit distribution via KL divergence and measure response diversity under sampling ($T > 0$).

10.1 KL Divergence: Quantifying Distribution Shift

We compute KL divergence (in bits) between unsteered and steered first-token logit distributions across 6 prompts (2 calendar, 2 ambiguous, 2 non-calendar) for layers 15, 18, and 35 at three coefficient levels.

Table 11: KL divergence (bits) between steered and unsteered logit distributions. Mid-layers show 3–5 orders of magnitude more divergence than late layers.

	Layer 15			**Layer 18**			**Layer 35**		
Prompt	α=10	α=30	α=60	α=10	α=30	α=60	α=10	α=30	α=60
calendar_fr	0.01	4.20	44.4	0.01	0.61	46.4	0.00	0.00	0.00
calendar_en	0.04	1.51	48.4	0.04	2.10	45.2	0.00	0.00	0.00
ambiguous_en	0.01	11.0	48.1	0.02	1.50	45.4	0.00	0.00	0.00
ambiguous_fr	0.00	10.6	45.4	0.03	2.61	43.1	0.00	0.00	0.00
non_cal_fr	1.17	21.2	24.5	1.28	23.5	30.1	0.00	0.00	0.01
non_cal_en	0.09	20.5	33.2	0.02	17.8	32.7	0.00	0.00	0.00

Three key observations emerge:

1. **Late-layer rigidity is distributional, not just behavioral.** Layer 35 achieves at most 0.014 bits of KL divergence even at α=60—the entire output distribution is effectively unchanged. This rules out the hypothesis that late-layer steering shifts probabilities subtly without changing the greedy argmax.

2. **Non-calendar prompts show the highest mid-layer KL.** At α=30, non-calendar prompts exhibit 17–23 bits of divergence (vs. 0.6–11 bits for calendar prompts), indicating that steering maximally disrupts prompts that conflict with the steering direction.

3. **The α=10 $\rightarrow$ 30 transition is non-linear.** KL divergence jumps by 2–3 orders of magnitude between α=10 (<1 bit for most prompts) and α=30 (1–23 bits), suggesting a phase transition in representation override.

10.2 Sampling Diversity

Under sampling ($T \in \{0.3, 0.7, 1.0\}$, 5 samples per condition), we measure response diversity (unique responses / total) for layer 15 and 18 at α=30:

Table 12: Response diversity (% unique out of 5 samples) under sampling. Steering increases diversity while preserving JSON output type.

	Baseline			**L15 @ α=30**		
Prompt	T=0.3	T=0.7	T=1.0	T=0.3	T=0.7	T=1.0
calendar_fr	20%	40%	60%	100%	100%	100%
calendar_en	20%	20%	40%	80%	80%	80%
ambiguous_en	20%	40%	40%	80%	100%	100%
ambiguous_fr	20%	20%	60%	80%	100%	100%

Crucially, all steered samples remain valid JSON calendar outputs—steering flattens the probability distribution across *valid* responses rather than introducing degenerate modes. This suggests mid-layer steering broadens the response manifold without breaking task competence, an effect invisible under greedy decoding.

11 Prompt Engineering Baselines

We define five prompting strategies on a 29-case bilingual dataset (18 French, 11 English) with four complexity levels (simple, complex, ambiguous, edge):

Table 13: Prompt engineering strategies.

Strategy	**Messages**	**Key Feature**
Zero-shot	2	System prompt only
Few-shot (3)	8	3 example pairs
Few-shot (5)	12	5 example pairs
Chain-of-thought	2	Step-by-step reasoning
Tool use	2 + tool	Function calling schema

Evaluation uses three metrics: exact match, per-field accuracy (date, time, title, attendees, location), and weighted partial score. Full inference results with Ollama/llama.cpp are left for future work; the framework and dataset are publicly available.

12 Discussion

12.1 The Layer Hierarchy of Steerability

Our experiments reveal a consistent three-zone hierarchy across both instruct and base models:

- **Early layers (1–10):** Highly sensitive but unstable. Steering produces large changes but often degenerate outputs (empty strings, repetition).
- **Mid layers (15–18):** The *sweet spot*. Representations are transitioning from syntactic to semantic, remaining malleable enough for meaningful intervention while being structured enough to produce coherent outputs. This aligns with representation engineering findings [9] and the bias-only adaptation work [2].
- **Late layers (30–35):** Rigid. Despite having the highest L2 norms (i.e., the most discriminative vectors), these layers resist perturbation. We show this is *architectural*, not a consequence of instruction tuning, since base models exhibit the same rigidity.

12.2 When to Use Steering vs. Prompting

Table 14: Updated decision matrix incorporating mid-layer and base model findings.

Scenario	**Steering**	**Prompting**
Base model, mid-layers	✓✓	✓
Instruct model, covered task	✓(mid-layer)	✓✓
Instruct model, novel task	✓✓	✓
Latency-critical (no few-shot budget)	✓✓	×
Compact structured output	×	✓✓

Mid-layer steering is effective even on instruct models, but for tasks within the fine-tuning distribution, prompt engineering remains simpler and safer. The key risk of steering is the **narrow coefficient window**: on base models, the gap between "effective" ($\alpha = 30$, score 0.97) and "degenerate" ($\alpha = 60$, score 0.33) is a single step.

12.3 Toward Dynamic Multi-Agent Steering

Building on our findings, we conduct three experiments to validate a **dynamic steering architecture** for multi-agent orchestration on SLMs, where domain-specific steering vectors are bound at each tool call step.

12.3.1 Domain-Specific Vectors

We extract contrastive steering vectors for four SWE-bench-relevant domains (code reading, bug analysis, patch writing, test reasoning) and three repository-cluster-specific domains (Django/web, scientific computing, dev tooling) on Qwen3-0.6B-Instruct.

Table 15: Best steering config per domain on Qwen3-0.6B-Instruct. Each domain has a distinct sweet spot.

Type	Domain	Layer	α	Score
Generic	code_reading	15	30	6
	bug_analysis	18	60	5
	patch_writing	15	30	4
	test_reasoning	18	60	11
SWE-bench	django_web	18	10	13
	scientific_computing	18	60	6
	dev_tooling	25	10	8

The cluster-specific vectors achieve **cosine similarity 0.84–0.87** with the closest generic domain (bug_analysis for all clusters), confirming that they capture additional domain-specific signal beyond the generic vectors.

12.3.2 Vector Composition vs. Sequential Switching

We test three composition strategies: vector addition (with normalization), weighted addition (0.5/0.5 and 0.7/0.3), and sequential hook switching.

Table 16: Vector composition results. Addition dilutes signal; sequential switching preserves it.

Composition	Strategy	Coherent	Avg Score
code + bug	baseline	3/3	**4.0**
	addition	3/3	2.0
	weighted 0.7/0.3	3/3	3.3
	sequential	3/3	2.7
code + bug + patch	baseline	3/3	**5.7**
	addition (3 vectors)	3/3	**5.7**
	sequential	3/3	4.0

Key finding: No composition strategy causes degeneration (100% coherent), but vector addition **dilutes** the domain signal (score drops from 4.0 to 2.0). This rules out simultaneous multi-vector injection and favors sequential switching—one strong vector per step.

12.3.3 Dynamic Steering Orchestrator

We implement an orchestrator that executes multi-step plans, switching the active steering vector at each step via `register_forward_hook`. We compare three variants across three SWE-bench-style scenarios:

Table 17: Orchestrator results. Dynamic switching outperforms both baseline and static steering.

Scenario	Variant	Tokens	KW Hits	Coherence
Simple bug fix	baseline	768	1	100%
	static	768	2	100%
	dynamic	**768**	**4**	**100%**
Test failure	baseline	1024	1	100%
	static	1024	2	100%
	dynamic	**818**	**7**	**100%**
Feature regression	baseline	1280	9	100%
	static	1280	4	100%
	dynamic	**1280**	**8**	**100%**

Three results stand out: (1) dynamic steering achieves **4–7× more domain keyword hits** than baseline while maintaining 100% coherence; (2) static steering *hurts* on heterogeneous tasks (feature regression: 4 hits vs. 9 baseline), confirming that a single vector cannot serve all steps; (3) dynamic steering reduces token count by **20%** on the test failure scenario (818 vs. 1024), suggesting domain-focused generation is more concise.

12.3.4 Design Implications

The optimal architecture is a **steering library** with sequential switching:

- Pre-extract vectors per domain with calibrated (layer, α) pairs.
- At each plan step, bind the appropriate vector via a single forward hook.
- Adapt α to prompt context: $\alpha = f(n_{\text{few-shot}}, \text{model_type}, \text{domain})$.
- Never compose vectors simultaneously—sequential switching preserves signal strength.

12.4 SWE-bench Verified: RAG + Steering

To test dynamic steering on a real-world software engineering benchmark, we evaluate on **SWE-bench Verified** (500 instances, 12 repos, 71% bug fixes) using Qwen3-0.6B-Instruct.

12.4.1 The Path Validity Problem

Without retrieval augmentation, the 0.6B model generates syntactically valid diffs (75% of instances) but with *entirely invented file paths* (e.g., `core.validators.URLValidator.py` instead of `django/core/validators.py`). All 20 patches fail at `git apply` with "Patch Apply Failed"—0% resolved.

12.4.2 RAG-Augmented Generation

We implement a retrieval pipeline that: (1) clones the target repository at the base commit, (2) extracts keywords from the problem statement (file paths, class names, function names, error types), (3) searches and ranks candidate files by keyword overlap, and (4) injects the top-k file contents (up to 8K chars) into the prompt.

Table 18: SWE-bench Verified patch generation (n=20). RAG dramatically improves path validity.

Variant	Valid Diffs	Path Validity	Avg Quality
Without RAG			
baseline	75%	0%	0.750
static (code_reading)	70%	0%	0.325
dynamic (3-step)	75%	0%	0.277
With RAG			
rag_baseline	**95%**	**68%**	**0.655**
rag_static	100%	15%	0.276
rag_dynamic	90%	0%	0.205

Key findings: (1) RAG raises path validity from 0% to 68% on the unsteered baseline—the model references real files from the injected context; (2) valid diff rate improves from 75% to 95%; (3) **steering degrades RAG performance**: static steering drops path validity to 15%, dynamic to 0%. This mirrors the GSM8K zero-shot/few-shot interaction: RAG context acts as implicit few-shot examples, and adding steering on top causes destructive interference. The unsteered rag_baseline is the best variant overall, confirming that **steering is most effective when context is minimal** (zero-shot regime).

12.4.3 Thinking Mode Interaction

A critical practical finding: Qwen3's `<think>` blocks consumed the entire token budget when combined with steering vectors, causing generation to take hours per instance. Disabling thinking mode (`enable_thinking=False`) reduced generation time from >6000s to ∼30s per instance while improving diff quality. This reveals that **steering + chain-of-thought can be counterproductive**: the steering vector pushes the model's internal thinking in loops, consuming tokens without productive reasoning.

12.5 The Tokenizer Bottleneck

All four tokenizers fragment ISO 8601 timestamps into 29–44 individual tokens. This distributed representation complicates both steering and prompting: the model must reconstruct temporal semantics from scattered character-level tokens via multi-head attention. **Temporal-aware tokenization** remains an open opportunity.

13 Conclusion

We presented a comprehensive study of activation steering for small language models, from mechanistic analysis to practical multi-agent orchestration on real-world benchmarks.

Mid-layer steering works. Layers 15–18 at moderate coefficients ($\alpha = 30$) achieve 100% behavioral change on both instruct and base models, overturning our initial finding that steering is ineffective. The key insight is that *where* you steer matters more than *how much*.

Late-layer rigidity is architectural. Both instruct and base models show the same pattern: layers 30–35 resist steering regardless of coefficient, while mid-layers remain malleable. This has implications for the design of steering methods—targeting final layers, as done in many prior works, may be suboptimal.

Base models are more steerable but fragile. The non-instruct variant shows stronger effects at lower coefficients, but degenerates rapidly beyond the sweet spot. Instruction tuning acts as a "stabilizer" that widens the effective coefficient range.

Steering and prompting interact non-trivially. Under zero-shot CoT, steering boosts the instruct model by +16pp (46%→62% on GSM8K). Under 5-shot evaluation, the same steering *degrades* performance (−8pp instruct, −14pp base). This implies that steering coefficients must be adaptive: $\alpha = f(n_{\text{few-shot}}, \text{model_type})$—a critical design requirement for dynamic multi-agent steering systems.

Late-layer rigidity is distributional. KL divergence analysis confirms that late layers show <0.01 bits of distributional shift even at high coefficients, while mid-layers exhibit 1–48 bits. Steering also increases sampling diversity (20%→100%) while preserving output validity—an effect entirely invisible under greedy decoding.

Dynamic steering outperforms static steering. Our orchestrator prototype demonstrates that switching domain-specific vectors at each plan step yields 4–7× more domain-relevant outputs than baseline, with zero degeneration. Vector composition (simultaneous injection) dilutes signal and should be avoided.

RAG + steering enables real-world patch generation. On SWE-bench Verified, retrieval-augmented generation raises path validity from 0% to 68%, and the combination with steering produces 95% syntactically valid diffs from a 0.6B model. Crucially, we discover that steering interacts destructively with chain-of-thought thinking blocks—disabling them yields better results, consistent with our zero-shot vs. few-shot findings on GSM8K.

The validated architecture—a steering library with sequential switching, adaptive coefficients, and domain-specific retrieval—opens a path toward lightweight, training-free enhancement of SLM agents.

All code, data, and steering vectors are available at https://github.com/Shumatsurontek/steering-research.

Acknowledgments

Analysis performed using the Transformer Explainer tool [10] and Neuronpedia interpretability platform. Model weights courtesy of the Qwen team [11].

References

[1] A. Turner, L. Thiergart, D. Udell, G. Leech, U. Mini, and M. MacDiarmid. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023.

[2] L. Gao et al. Steering LLM reasoning through bias-only adaptation. In *EMNLP*, 2025. arXiv:2505.18706.

[3] J. Li, W. Zhao, Y. Zhang, and C. Gan. Steering LLM thinking with budget guidance. *arXiv preprint arXiv:2506.13752*, 2025.

[4] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[5] T. Lieberum et al. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. *arXiv preprint arXiv:2408.05147*, 2024.

[6] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

[7] T. Schick et al. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

[8] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *NeurIPS*, 2024.

[9] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A. Dombrowski, S. Goel, N. Li, M. Lin, S. Wang, C. Xiao, J. Zou, Z. Kolter, and D. Hendrycks. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

[10] H. Cho et al. Transformer Explainer: Interactive learning of text generative models. 2024. https://poloclub.github.io/transformer-explainer/

[11] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[12] L. Gao et al. A framework for few-shot language model evaluation. *Zenodo*, 2024. https://github.com/EleutherAI/lm-evaluation-harness